Introdução ao Lean 4 e à Verificação Formal de Provas

Dia 2: Axiomas de Peano, Indução e Provas ao Vivo

Departamento de Matemática e Matemática Aplicada
Universidade Federal de Lavras

Minicurso — 2026

Roteiro do Dia 2

1. Os números naturais no Lean: axiomas de Peano, tipo indutivo e adição
2. Táticas e estratégia para resolver problemas
3. Atividades guiadas: da prova no papel à prova no Lean
4. Desafios: agora é com vocês!
5. Onde ir a seguir

Os Números naturais: Axiomas de Peano

Axiomas de Peano — versão moderna

Seja $\mathbb{N}$ um conjunto cujos elementos são chamados de **números naturais**, e seja $S : \mathbb{N} \to \mathbb{N}$ uma função **sucessor**. Os axiomas de Peano são:

1. **Existência do zero:** $0 \in \mathbb{N}$.
2. **Todo número tem um sucessor:** Para todo $n \in \mathbb{N}$, existe um único sucessor $S(n) \in \mathbb{N}$.
3. **Zero não é sucessor de ninguém:** Não existe $n \in \mathbb{N}$ tal que $S(n) = 0$.
4. **Injetividade da função sucessor:** Se $S(m) = S(n)$, então $m = n$ (para quaisquer $m, n \in \mathbb{N}$).
5. **Princípio da Indução Matemática:** Seja $P(n)$ uma propriedade sobre $n \in \mathbb{N}$. Se:
   (i) $P(0)$ é verdadeira (passo base), e
   (ii) para todo número natural k, se $P(k)$ é verdadeira então $P(S(k))$ é verdadeira (passo indutivo),

   então $P(n)$ é verdadeira para todo $n \in \mathbb{N}$.

Axiomas de Peano

Os axiomas 3, 4 e 5 são *automáticos* para tipos indutivos no Lean.
O recursor (indução) é gerado pelo compilador.

Preparação para a sessão ao vivo

Ambiente

`live.lean-lang.org`

- Sem instalação. Abra no navegador agora.
- O Lean 4 com Mathlib roda no servidor.
- O painel direito (*Infoview*) mostra o estado das provas.

Cole este cabeçalho para começar

```
import Mathlib.Data.Nat.Basic
```

Os Números naturais: definição indutiva

Tipo indutivo

```
inductive Nat : Type where
  | zero : Nat
  | succ (n : Nat) : Nat
```

Adição de Peano no Lean

A adição é definida *por recursão* no segundo argumento:

$$\begin{array}{cc} n + 0 = n & n + \mathrm{suc}(m) = \mathrm{suc}(n + m) \\ \downarrow & \downarrow \\ n + 0 = n & n + (m + 1) = (n + m) + 1 \end{array}$$

```
def add : Nat -> Nat -> Nat
  | n, .zero    => n                    -- n + 0 = n
  | n, .succ m => .succ (add n m)       -- n + suc m = suc (n + m)
```

Por que isso importa para as provas?

- `n + 0 = n` é *verdadeiro por definição* (caso base)
- `n + succ m = succ (n + m)` é *verdadeiro por definição* (caso recursivo)
- `rfl` prova esses enunciados diretamente
- `0 + n = n` **não** é verdadeiro por definição — requer indução!

Táticas principais que usaremos

Tática NNG	**O que faz**	**Quando usar**
`rfl`	Fecha objetivo por igualdade definitória	"ambos os lados são idênticos"
`rewrite`[h]	Apenas reescreve da esq. para dir. usando *h*	substituição direta
`rw [h]`	Faz o mesmo que `rewrite[h]` + tenta rodar `rfl`	substituição direta + verificação de igualdade
`rw [← h]`	Reescreve da dir. para esq.	substituição inversa
`simp`	Simplifica com lemas automáticos	simplificação algébrica
`simp [h]`	Simplificação automática com *h*	simplificações encadeadas

Táticas principais que usaremos

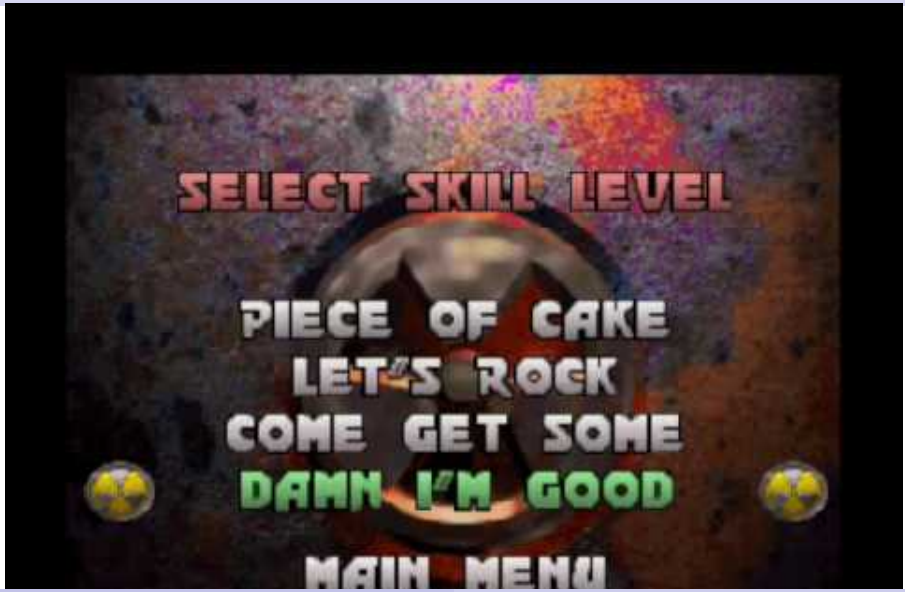
Tática NNG	O que faz	Quando usar
`ring`	Identidades em semianéis comutativos	álgebra pura
`induction n`	Abre caso base e passo indutivo	provas por indução
`cases n`	Abre caso $n = 0$ e $n = \mathrm{suc}\, k$	análise de casos
`intro h`	Introduz hipótese h	provar $P \Rightarrow Q$
`apply h`	Reduz objetivo usando $h : A \Rightarrow B$	modus ponens
`exact`	Fornece o termo de prova diretamente	q.e.d.
`exact h`	Fecha objetivo com h exato	q.e.d.
`have`	Introduz um lema auxiliar ou hipótese intermediária	Introduz uma afirmação

Importante: **táticas** são diferentes de **nomes de lemas** (`mul_assoc` é um lema; `rw [mul_assoc]` é uma tática usando esse lema)

Sugestão de estratégia para resolução de problemas no Lean

1. Leia o enunciado. Reconheça sua forma lógica.
2. Pergunte: *isso é verdadeiro por definição?* $\rightarrow$ tente `rfl`
3. É uma identidade algébrica? $\rightarrow$ tente `ring`
4. Envolve algum n arbitrário com recursão? $\rightarrow$ pense em `induction`
5. Veja quais teoremas já estão disponíveis e tente `rw` ou `rewrite` com eles

Mãos na massa: Atividades e Desafios



Atividade 1 — 2+2=4 (Piece of cake!)

Usando os axiomas de Peano, mostre que

$$2 + 2 = 4$$

$$n + 0 = n, \qquad n + \mathrm{suc}(m) = \mathrm{suc}(n + m)$$

Prova em linguagem natural:

- $4 = \mathrm{suc}(\mathrm{suc}(\mathrm{suc}(\mathrm{suc}(0))))$
- $2 = \mathrm{suc}(\mathrm{suc}(0))$

$$\begin{aligned} 2 + 2 &= \mathrm{suc}(\mathrm{suc}(0)) + \mathrm{suc}(\mathrm{suc}(0)) \\ &= \mathrm{suc}(\mathrm{suc}(\mathrm{suc}(0)) + \mathrm{suc}(0)) \\ &= \mathrm{suc}(\mathrm{suc}(\mathrm{suc}(\mathrm{suc}(0)) + 0)) \\ &= \mathrm{suc}(\mathrm{suc}(\mathrm{suc}(\mathrm{suc}(0)))) \\ &= 4 \end{aligned}$$

Atividade 1 — 2+2=4 (Piece of cake!)

Enunciado (NNG)

$\vdash$ 2 + 2 = 4

Em Lean:

```
example : (2 : Nat) + 2 = 4 := by
  rfl   -- o kernel computa e verifica
        diretamente
```

Ponto pedagógico

`rfl` funciona porque o Lean *avalia* ambos os lados e compara. (Não é uma "mágica" :-\)

Atividade 1 — 2+2=4 (Let's rock!)

Usando algumas táticas e teoremas disponíveis é possível reproduzir em Lean a demonstração de linguagem natural. Neste caso,

- Tática: `rewrite`
- Teorema: Para todo $n, m \in \mathbb{N}$, $n + (m + 1) = (n + m) + 1$.

No Lean:

```
theorem add_succ (n m : Nat) : n + succ m = succ (n + m) :=
    rfl
```

- Teorema: Para todo $n \in \mathbb{N}$, $n + 0 = n$.

No Lean:

```
theorem Nat.add_zero (n : Nat) : n + 0 = n := rfl
```

Importante: Teorema é satisfeito por definição, por essa razão usamos `rfl`.

Atividade 1 — 2+2=4 (Let's rock!)

Em Lean:

```
example : (2 : Nat) + 2 =4 := by
  -- 1. Reescrevendo 2 e 4 usando sucesores
   have h2 : (2 : Nat) = Nat.succ (Nat.succ Nat.zero) := by rfl
   have h4 : (4 : Nat) = Nat.succ (Nat.succ (Nat.succ (Nat.succ Nat.zero))
       ) := by rfl
  -- 2. Substituindo os numeros pelas definicoes em succ usando rewrite
   rewrite [h2 , h4]
   rewrite[Nat.add_succ]
   rewrite[Nat.add_succ]
   rewrite[Nat.add_zero]
   rewrite[h2]
   -- Obj: Nat.zero.succ.succ.succ.succ = Nat.zero.succ.succ.succ.succ
   rfl
```

Atividade 2 — Sucessor de um número (Piece of Cake!)

Enunciado (Lean)

Prove que `succ n = n + 1` para todo `n : Nat`.

Atividade 2 — Sucessor de um número (Piece of Cake!)

Enunciado (Lean)

Prove que `succ n = n + 1` para todo `n : Nat`.

Em Lean:

```
theorem succ_eq_add_one (n : Nat) : n.succ = n + 1 := by
  rfl
```

Atividade 2 — Sucessor de um número (Piece of Cake!)

Enunciado

Prove que `succ n = n + 1` para todo $n \in \mathbb{N}$.

$$n + 0 = n, \qquad n + \mathrm{suc}(m) = \mathrm{suc}(n + m)$$

Prova em linguagem natural:

- Por definição, $1 = \mathrm{suc}(0)$

$$\begin{aligned} n + 1 &= n + \mathrm{suc}(0) \\ &= \mathrm{suc}(n + 0) \\ &= \mathrm{suc}(n) \end{aligned}$$

Atividade 2 — Sucessor de um número (Let's Rock!)

Enunciado (Lean)

Prove que `succ n = n + 1` para todo `n : Nat`.

Em Lean:

```
theorem succ_eq_add_one (n : Nat) : n.succ = n + 1 := by
--1. Indroduzimos uma hipotese intermediaria: Escrever 1 como sucessor de
    zero
    have h : (1 : Nat) = Nat.succ 0 := by rfl
    rewrite[h]
    rewrite[Nat.add_succ]
    rewrite[Nat.add_zero]
    rfl
```

Atividade 3 — Zero soma (Piece of Cake!)

Enunciado

Prove que $0 + n = n$ para todo $n \in \mathbb{N}$.

Prova em linguagem natural (por indução em n):

- **Caso Base** ($n = 0$): $0 + 0 = 0$ ✓
- **HI**: Suponha $0 + n = n$
- **Prove** $0 + (n + 1) = n + 1$:

$$\begin{aligned} 0 + (n + 1) &= (0 + n) + 1 \\ \text{(por hipótese de indução)} &= n + 1 \checkmark \end{aligned}$$

Atividade 3 — Zero soma (Let's Rock!)

Enunciado (Lean)

Prove que `0 + n = n` para todo `(n : Nat)`. (Obs!: Não pode usar a comutativaidade da soma)

Atividade 3 — Zero soma (Let's Rock!)

Enunciado (Lean)

Prove que `0 + n = n` para todo `(n : Nat)`. (Obs!: Não pode usar a comutativaidade da soma)

Por que não basta `rfl`?: A adição é recursiva no segundo argumento. `0 + n` não reduz diretamente.

Atividade 3 — Zero soma (Let's Rock!)

Zero soma – Lean

```
theorem zero_add (n : Nat) : 0 + n = n := by
    induction n with
        | zero => -- 0 + 0 = 0
            rfl
        | succ n ih => -- ih : 0 + n = n, Objetivo Atual!: 0 + (n + 1) = n
            + 1
          rw[Nat.add_succ] -- 0 + (n + 1) = n + 1 => succ (0 + n) = n + 1
          rw[ih] --succ (0 + n) = n + 1 => succ n = n+1
          --(Aqui! usar "rw" ja aplica "rfl". Ou seja, Lean ja entende a
              ultima igualdade como verdadeira)
          -- Obs: usando rewrite[ih] depois precisa usar rfl
```

Atividade 4 — Sucessor da adição (Let's Rock!)

Enunciado

Prove que `suc(n)+m=suc(n+m)` para todo `n,m : Nat`

Propriedades conhecidas!

A adição é definida *por recursão* no segundo argumento:

$$\begin{array}{ccc} n+0=n \ \ (*), & & n+\mathrm{suc}(m)=\mathrm{suc}(n+m) \ \ (**) \\ \downarrow & & \downarrow \\ n+0=n, & & n+(m+1)=(n+m)+1 \end{array}$$

Atividade 4 — sucessor da adição (Let's Rock!)

Enunciado

Prove que `suc(n)+m=suc(n+m)` para todo `n,m : Nat`

Prova em linguagem natural (indução em m):

- **Caso Base** ($m = 0$): $(n + 1) + 0 = n + 1 = (n + 0) + 1$
- **HI**: Suponha que $(n + 1) + m = (n + m) + 1$
- **Passo**: Vejamos que $(n + 1) + (m + 1) = (n + (m + 1)) + 1$:
  - Usando a Propriedade da adição (**), segue que: $(n + 1) + (m + 1) = ((n + 1) + m) + 1$
  - Usando HI, $((n + 1) + m) + 1 = ((n + m) + 1) + 1$. Ou seja,

$$(n + 1) + (m + 1) = ((n + m) + 1) + 1 \tag{1}$$

  - Por outro lado, usando novamente a Propriedade da adição (**), segue que:

$$(n + (m + 1)) + 1 = ((n + m) + 1) + 1 \tag{2}$$

Atividade 4 — Sucessor da adição (Let's Rock!)

Enunciado (Lean)

Prove que `suc(n)+m=suc(n+m)` para todo `n,m : Nat`

```
theorem suc_add (n m : Nat) : Nat.succ n + m = Nat.succ (n + m) := by
    induction m with
        | zero =>
            rw[Nat.add_zero]
        | succ m ih =>
            rewrite[Nat.add_succ]
            rewrite[ih] --Ate aqui seria a deducao de (1)
            rewrite[Nat.add_succ] --aqui seria a deducao de (2)
            rfl -- equivalente a "Segue de (1) e (2) que"
```

Atividade 5 — Comutatividade da adição (Let's Rock!)

Enunciado

Prove usando indução que `n + m = m + n` para todo `n m : Nat`.

Prova em linguagem natural (indução em n):

- **Caso Base** ($n = 0$): $0 + m = m = m + 0$
- **HI**: Suponha que $n + m = m + n$
- **Passo**: Prove $\mathrm{suc}(n) + m = m + \mathrm{suc}(n)$:

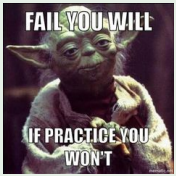
$$\begin{aligned} \mathrm{suc}(n) + m &= \mathrm{suc}(n + m) && \text{(lema auxiliar } \texttt{succ_add}\text{)} \\ &= \mathrm{suc}(m + n) && \text{(hip. indução)} \\ &= m + \mathrm{suc}(n) && \text{(por } \texttt{add_succ}\text{)} \checkmark \end{aligned}$$

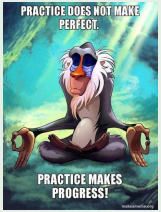
Desafio 1 — Comutatividade da adição (Let's Rock!)

```
theorem add_comm (n m : Nat) : n + m = m + n := by
```

No Lean

Chegou sua Hora!





Observe a hierarquia

`add_comm` depende de `zero_add`, `add_zero`, `succ_add`, `add_succ`.

Cada prova constrói sobre as anteriores — exatamente como em um livro de álgebra.

Desafio 2 — Associatividade da soma (Come Get Some!)

Complete a demonstração no Lean

```
theorem add_assoc (n m k : Nat) : n + m + k = n + (m + k) := by
  induction k with
  | zero      => sorry  -- use add_zero
  | succ k ih => sorry  -- use add_succ e ih
```

Obs.: Parênteses invisíveis

Quando o Lean escreve `a + b + c`, ele quer dizer `(a + b) + c`.

Desafio 3 — Propriedade distributiva (Let's Rock!)

Propriedade distributividade

Prove usando indução que $a \cdot (b + c) = a \cdot b + a \cdot c$ para $a, b, c \in \mathbb{N}$.

Lema auxiliar que você precisará (já em Mathlib):

- `Nat.mul_succ`: $\forall\ nm \in \mathbb{N},\ n * m.succ = n * m + n$
- `Nat.mul_zero` : $\forall\ n \in \mathbb{N},\ n * 0 = 0$

No Lean

```
theorem meu_distrib (a b c : Nat) : a * (b + c) = a * b + a * c := by
  induction c with
    | zero      =>
      simp [Nat.mul_zero, Nat.add_zero] --alternativa: pode utilizar rw[]
    | succ c ih =>
```

Desafio 3— Propriedade distributiva (Let's Rock!)

Continuação

```
rw [Nat.add_succ]
rw [Nat.mul_succ]
rw [ih]
rw[Nat.mul_succ]
rw [<-Nat.add_assoc]
```

← Nat.add_assoc

`Nat.add_assoc` : $n + m + k = n + (m + k)$ Lean entende $n + m + k \Rightarrow n + (m + k)$.
Quando colocar ← Lean entende a regra invertida $n + (m + k) \Rightarrow n + m + k$ (ou
"$n + (m + k) = n + m + k$").

Desafio 4 — soma de dois ímpares é par (Damn I'm Good!)

Demonstração em Linguagem natural:

Definição de número par e ímpar:

- Um **número par** é qualquer número que pode ser escrito da forma `x + x`
- Um **número ímpar** é definido como `S(x + x)`

Propriedades: (assumidas!)

1. `n + 0 = n`
2. `n + S(m) = S(n + m)`
3. `S(n) + m = S(n + m)`
4. `n + m = m+ n`
5. `(n + m) + p = n + (m + p)`

Desafio 4 — soma de dois ímpares é par (Damn I'm Good!)

Demonstração em linguagem natural:

$$A = s(x+x) \text{ e } B = s(y+y)$$

$$\begin{aligned}
A + B = s(x+x) + s(y+y) &\Rightarrow s(x+x) + s(y+y) = s(s(x+x) + (y+y)) \text{ (Por 2.)} \\
\text{(Por 4.)} &\Rightarrow s(s(x+x) + (y+y)) = s((y+y) + s(x+x)) \\
\text{(Por 2.)} &\Rightarrow s((y+y) + s(x+x)) = s(s((y+y) + (x+x))) \\
\text{(Por 5.)} &\Rightarrow s(s((y+y) + (x+x))) = s(s(y + (y + (x+x)))) \\
\text{(Por 5.)} &\Rightarrow s(s(y + (y + (x+x)))) = s(s(y + ((y+x) + x))) \\
\text{(Por 4.)} &\Rightarrow s(s(y + ((y+x) + x))) = s(s(y + (x + (y+x)))) \\
\text{(Por 3.)} &\Rightarrow s(s(y + (x + (y+x))) = s((y + x + s(y+x))) \\
\text{(Por 2.)} &\Rightarrow s((y + x + s(y+x))) = s(x + y + s(x+y)) \\
\text{(Por 5.)} &\Rightarrow s(s(y + (x + (y+x)))) = s(s((y+x) + (y+x))) \\
\text{(Por 2.)} &\Rightarrow s(s((y+x) + (y+x))) = s\big((y+x) + s(y+x)\big)
\end{aligned}$$

Desafio 4 — soma de dois ímpares é par (Damn I'm Good!)

Soma de dois números ímpares é par

Prove que a soma de dois números ímpares é um número par.

Lema auxiliar que você precisará (já em Mathlib):

- `Nat.mul_succ`: $\forall\ nm \in \mathbb{N},\ n * m.succ = n * m + n$
- `Nat.mul_zero` : $\forall\ n \in \mathbb{N},\ n * 0 = 0$
- $\leftarrow$ `Nat.add_assoc`: $n + (m + k) = n + m + k = (n + m) + k$

Coisas mais avançadas que poderíamos fazer!

Cálculo diferencial—Continuidade de funções

Se f, g funções contínuas num ponto $a \in \mathbb{R}$, então a soma $(f+g)$, dada por $(f+g)(x) = f(x) + g(x)$, é contínua no ponto a.

```
theorem continuousAt_add (f g : Real -> Real) (a : Real) (hf :
    ContinuousAt f a) (hg : ContinuousAt g a) : ContinuousAt (fun x => f x
    + g x) a := by
```

Coisas mais avançadas

```
import Mathlib.Analysis.Normed.Group.Basic
open Real

theorem continuousAt_add (f g : ℝ → ℝ) (a : ℝ) (hf : ContinuousAt f a) (hg : ContinuousAt g a) : ContinuousAt (fun x
 => f x + g x) a := by
 rw [Metric.continuousAt_iff] at *
 -- hf : ∀ ε > 0, ∃ δ > 0, ∀ x, dist x a < δ → dist (f x) (f a) < ε
 -- hg : ∀ ε > 0, ∃ δ > 0, ∀ x, dist x a < δ → dist (g x) (g a) < ε
 intro ε hε
 obtain ⟨δ_1, hδ_1, h_1⟩ := hf (ε/2) (half_pos hε)   -- continuidade de f
 obtain ⟨δ_2, hδ_2, h_2⟩ := hg (ε/2) (half_pos hε)   -- continuidade de g
 refine ⟨min δ_1 δ_2, lt_min hδ_1 hδ_2, fun x hx => ?_⟩ -- δ = min(δ₁, δ_2)
 have hx₁ : dist x a < δ_1 := lt_of_lt_of_le hx (min_le_left _ _)
 have hx₂ : dist x a < δ_2 := lt_of_lt_of_le hx (min_le_right _ _)

--Temos que introduzir um par de lemas auxiliares para evitar problema no passo (*) embaixo

 have h_dist1 : dist (f x + g x) (f a + g x) = dist (f x) (f a) := by
   rw [Real.dist_eq, Real.dist_eq]
   congr 1
   ring

 have h_dist2 : dist (f a + g x) (f a + g a) = dist (g x) (g a) := by
   rw [Real.dist_eq, Real.dist_eq]
   congr 1
   ring
 calc
   dist (f x + g x) (f a + g a)
     ≤ dist (f x + g x) (f a + g x) + dist (f a + g x) (f a + g a) := dist_triangle _ _ _
   _ =   dist (f x) (f a) + dist (g x) (g a)                      := by rw [h_dist1 , h_dist2]--(*)
   _ < ε / 2 + ε / 2                                              := add_lt_add (h_1 hx_1) (h_2 hx_2)
   _ = ε   := add_halves ε
```

Algumas observações deste código

(1) `rw [Metric.continuousAt_iff] at *`

- `Metric.continuousAt_iff` : é o teorema

$$\texttt{ContinuousAt}\ fa \leftrightarrow\ \forall\ \varepsilon > 0,\ \exists\ \delta > 0,\ \forall\ x, dist xa < \delta \rightarrow dist(fx)(fa) < \varepsilon$$

- `at *`, significa aplicar (`rw[ ]`) tanto no seu Goal (o objetivo) quanto em todas as hipóteses listadas naquele momento. Neste caso, o teorema `Metric.continuousAt_iff` foi aplicado nas hipóteses `hf` e `hg`. Assim, essas hipóteses acabam sendo reescritas como:

$$hf\ :\quad \forall\quad \varepsilon > 0,\ \exists \delta > 0,\ \forall\ x,\ dist(x,\ a) < \delta \Rightarrow dist(f(x),\ f(a)) < \varepsilon$$
$$hg\ :\quad \forall\quad \varepsilon > 0,\ \exists \delta > 0,\ \forall\ x,\ dist(x,\ a) < \delta \Rightarrow dist(g(x),\ g(a)) < \varepsilon$$

Algumas observações deste código

(2) `intro` é a tática para introduzir a hipóteses $\mathtt{h}\varepsilon : \varepsilon > 0$

(3) `half_pos` $h\varepsilon : \ \varepsilon > 0 \Rightarrow \varepsilon/2 > 0$

(4) `congr` (abreviação de congruence) é uma tática que serve para reduzir uma meta de igualdade entre duas expressões complexas à igualdade de suas partes componentes. Em termos simples, para $(f(x) = f(y))$, a tática congr assume que a f é a mesma dos dois lados e transforma o seu objetivo em provar apenas que $x = y$. Ela funciona para qualquer função f, mesmo que você não saiba se ela é injetiva ou não no sentido estrito.

Resumo do Dia 2

A ideia central

Provar = construir um objeto de tipo correto.
O computador verifica. Você guia.

Recursos para continuar

Jogar e praticar

- **NNG completo**: `adam.math.hhu.de`
  Mundos: Addition, Multiplication, Power, Function, Proposition, Advanced
- **Lean Web**: `live.lean-lang.org`
  Para experimentar com Mathlib

Documentação

- **Theorem Proving in Lean 4**
  `leanprover.github.io/theorem_proving_in_lean4`
- **Mathlib4 docs**
  `leanprover-community.github.io/mathlib4_docs`

Livros

- **Functional Programming in Lean**
  `leanprover.github.io/functional_programming_in_lean`
- **Mathematics in Lean**
  `leanprover-community.github.io/mathematics_in_lean`

Comunidade

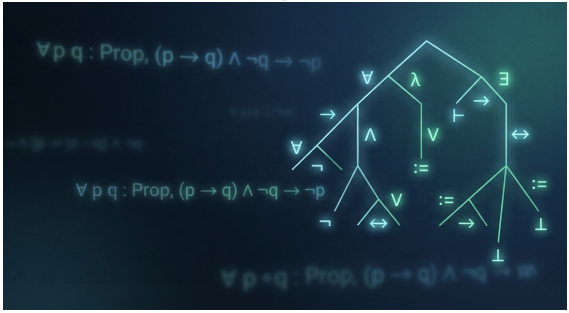
Zulip Lean: `leanprover.zulipchat.com`
Tópicos: `#new members`, `#mathlib4`
(a comunidade é muito receptiva a iniciantes)

Resumo do minicurso

O que construímos nesses 2 dias

1. Entendemos por que verificação formal é diferente de SAT/SMT
2. Vimos a correspondência de Curry–Howard: provas são termos tipados
3. Aprendemos a sintaxe básica do Lean 4 e o papel do LSP
4. Formalizamos os axiomas de Peano como tipo indutivo
5. Provamos resultados reais sobre $\mathbb{N}$ com indução no Lean
6. Escrevemos provas ao vivo no ambiente web, sem instalação

Obrigado!



Dúvidas, sugestões e colaborações são bem-vindas.

Agradecimentos!



Este minicurso foi desenvolvido no âmbito do projeto **Integração entre IA e Assistentes de Prova para Inovação na Pesquisa Matemática e em Processos Organizacionais (DMM/UFLA)**.

Coordenação: Prof. Ricardo J. Franquiz

Membros Docentes: Profa. Ana C. Dias do Amaral Ramos, Prof. Helvécio G. Fargnoli Filho, Prof. Marlon Pimenta Fonseca, Profa. Thais Presses Mendes

Bolsistas: Alan Gabriel Fernandes · Guilherme A. de Souza Candinho Discentes: Luiz G. Silva Prata